

6-1 · 실시간 데이터와 Kafka · 실습 2.5H

Kafka 핵심 구조와 첫 실행

토픽·파티션·오프셋을 잡고, 브로커를 띄워 첫 메시지

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

2.5시간 뒤에는

- ① 브로커·토픽·파티션·오프셋을 우체국 비유로 설명한다
- ② Docker 로 KRaft 단일 브로커 Kafka 를 기동한다
- ③ CLI 로 토픽을 만들고 메시지를 보내고 받는다
- ④ 읽어도 사라지지 않는 것과 「같은 키 = 같은 파티션」을 눈으로 확인한다

오늘의 kafka 컨테이너와 hello-stream 토픽은 개념용 — 6-2 부터 compose 로 정식 구성합니다

1. 기본 낱말

우체국으로 한 번에 — 편지는 읽어도 사라지지 않는다



브로커 = 우체국 건물 · 오프셋 = 파티션 안 일련번호 · KRaft = ZooKeeper 없이 스스로 메타데이터 관리

큐와 다른 점 하나 — Kafka 의 편지는 보존 기간 동안 그대로 남아, 여러 부서(컨슈머 그룹)가 각자 자기 속도로 같은 편지를 읽는다

1. 기본 낱말

순서는 파티션 안에서만 보장된다

1

같은 키 → 언제나 같은 파티션

키의 해시로 정해진다 — 한 파티션 안에서는 넣은 순서대로 읽힌다

2

파티션이 여러 개면 전체 순서는 없다

파티션끼리는 독립 — 「주문 ID별 순서」가 필요하면 주문 ID 를 키로

3

키를 안 주면 라운드로빈

골고루 흩어진다 — 순서가 필요 없을 때

전체 순서가 꼭 필요하다면

**파티션 1개
= 병렬 없음**

대부분의 업무는 「같은 대상 안에서의 순서」만 필요 — 키를 잘 고르는 것이 설계 · Spark 파티션처럼 병렬 처리의 단위

일곱 단계 — ~/data-eng/week6 에서 docker exec 로

터미널 두 개를 씁니다 — 하나는 Consumer 를 켜 두고, 다른 하나에서 Producer 로 입력합니다

1. 기동

apache/kafka
3.7.0

2. 토픽

hello-stream
파티션 3

3·4. 송수신

재시작해도
남아 있나

5·6. 키·오프셋

같은 키
같은 파티션

7. 미니 과제

파티션 1개
--group

1·2단계

핵심 명령 — 기동 · 확인 · 토픽

기동	<code>docker run -d --name kafka -p 9092:9092 apache/kafka:3.7.0</code>
뒀는지	<code>docker ps --filter name=kafka</code>
준비됐는지	<code>docker logs kafka grep -i "kafka server started"</code>
토픽 만들기	<code>kafka-topics.sh --create --topic hello-stream --partitions 3</code>
토픽 상세	<code>kafka-topics.sh --describe --topic hello-stream</code>

CLI 는 컨테이너 안 `/opt/kafka/bin` — 앞에 `docker exec kafka /opt/kafka/bin/` 과 `--bootstrap-server localhost:9092` 를 붙인다

핵심 명령 — 보내기 · 읽기 · 키 · 오프셋

Consumer	kafka-console-consumer.sh --topic hello-stream --from-beginning
Producer	kafka-console-producer.sh --topic hello-stream
키 함께 보내기	--property parse.key=true --property key.separator=:
키·파티션 출력	--property print.key=true --property print.partition=true
오프셋	kafka-run-class.sh kafka.tools.GetOffsetShell --topic hello-stream

console producer 는 한 줄이 메시지 하나 · A:첫 번째 / A:두 번째 / B:다른 키 / A:세 번째 순으로 입력해 본다

단계마다 확인 포인트 — 오늘의 핵심 관찰은 4단계

- 1 3. 터미널 2 입력이 터미널 1 에 곧바로**
Producer 한 줄 = 메시지 하나
- 2 4. Consumer 를 끄고 --from-beginning 으로 다시**
아까 읽은 메시지가 다시 전부 나온다 — 읽어도 사라지지 않는다
- 3 5. 키 A 의 세 메시지**
모두 같은 파티션 번호, 입력한 순서 그대로 — B 는 다른 파티션일 수 있다
- 4 6. hello-stream:0:N**
더 보내고 다시 실행하면 N 이 늘어난다 — 오프셋은 일련번호

--from-beginning 을 빼면

**지금부터만 —
새로 보낸 것만**

replication-factor 2 로 먼저 만들어 보기

「복제본 없음」이 무슨 뜻인지 실패로 확인합니다 — 기동 직후 CLI 를 바로 치면 Connection refused 도 함께 보게 됩니다

2 로 먼저	만들어지지 않는다	이유	1 로 다시
--replication-factor 2 hello-stream	오류 메시지를 읽어 본다	브로커 1대 — 복제본 둘 곳 없음	운영은 브로커 3대 복제 3

이 2.5시간이 끝날 때 확인할 것

- 1 **docker ps** 의 **STATUS** 가 **Up** · **PORTS** 에 **0.0.0.0:9092→9092/tcp**
- 2 **describe** 에 **PartitionCount: 3** · **Partition 0/1/2** · **Leader 1**
- 3 터미널 2 에 입력한 문장이 터미널 1 에 곧바로 나타난다
- 4 **Consumer** 를 **--from-beginning** 으로 다시 켜면 이전 메시지가 전부
- 5 키 **A** 의 세 메시지가 같은 파티션에, 입력한 순서대로
- 6 **GetOffsetShell** 의 **hello-stream:0:N** 이 보낼수록 늘어난다

6-1 · 실습 2.5H 마무리

다음은 정리·회고 (0.5H)

용어를 우체국 비유로 한 장에 정리하고, 6-2 클러스터 구성을 위한 준비를 확인합니다

✓ 용어 8개 · CLI 명령 7개 요약

✓ 자주 겪는 오류 네 가지

✓ `docker rm -f kafka` · 이미지는 남기기

✓ 메모리 8GB 이상 확인

가져갈 것

GetOffsetShell 이 찍은 파티션별 오프셋